Team Foundation Server

Guide Document

Purpose of document	Guidelines to be followed by Developers while working on Team Foundation Server
Target audience	Developers

Version #	Date	Contributor(s)	Editor(s)	Version details (what changed)
1.0	04/27/2022	Vishakha		Initial listing of points
2.0	07/19/2022	Vishakha		Detail description of the document
3.0	08/09/2022	Vishakha		Finalizing the sequence and adding remaining point

Table of Contents

A. What is Team Foundation Version Control?

Version control systems are software that help you track changes you make in your code over time. As you edit your code, you tell the version control system to take a snapshot of your files. The version control system saves that snapshot permanently so you can recall it later if you need it.

Azure DevOps Services and TFS provide two models of version control: Git, which is distributed version control, and Team Foundation Version Control (TFVC), which is centralized version control.

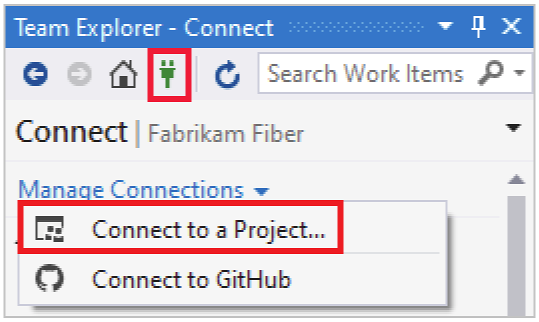
Version control systems provide the following benefits:

- Create workflows - Version control workflows prevent the chaos of everyone using their own development process with different and incompatible tools. Version control systems provide process enforcement and permissions, so everyone stays on the same page.
- Work with versions - Every version has a description for what the changes in the version do, such as fix a bug or add a feature. These descriptions help you follow changes in your code by version instead of by individual file changes. Code stored in versions can be viewed and restored from version control at any time as needed. This makes it easy to base new work off any version of code.
- Code together - Version control synchronizes versions and makes sure that your changes don't conflict with other changes from your team. Your team relies on version control to help resolve and prevent conflicts, even when people make changes at the same time.
- Keep a history - Version control keeps a history of changes as your team saves new versions of your code. This history can be reviewed to find out who, why, and when changes were made. History gives you the confidence to experiment since you can roll back to a previous good version at any time. History lets your base work from any version of code, such as to fix a bug in a previous release.
- Automate tasks - Version control automation features save your team time and generate consistent results. You can automate testing, code analysis, and deployment when new versions are saved to version control.

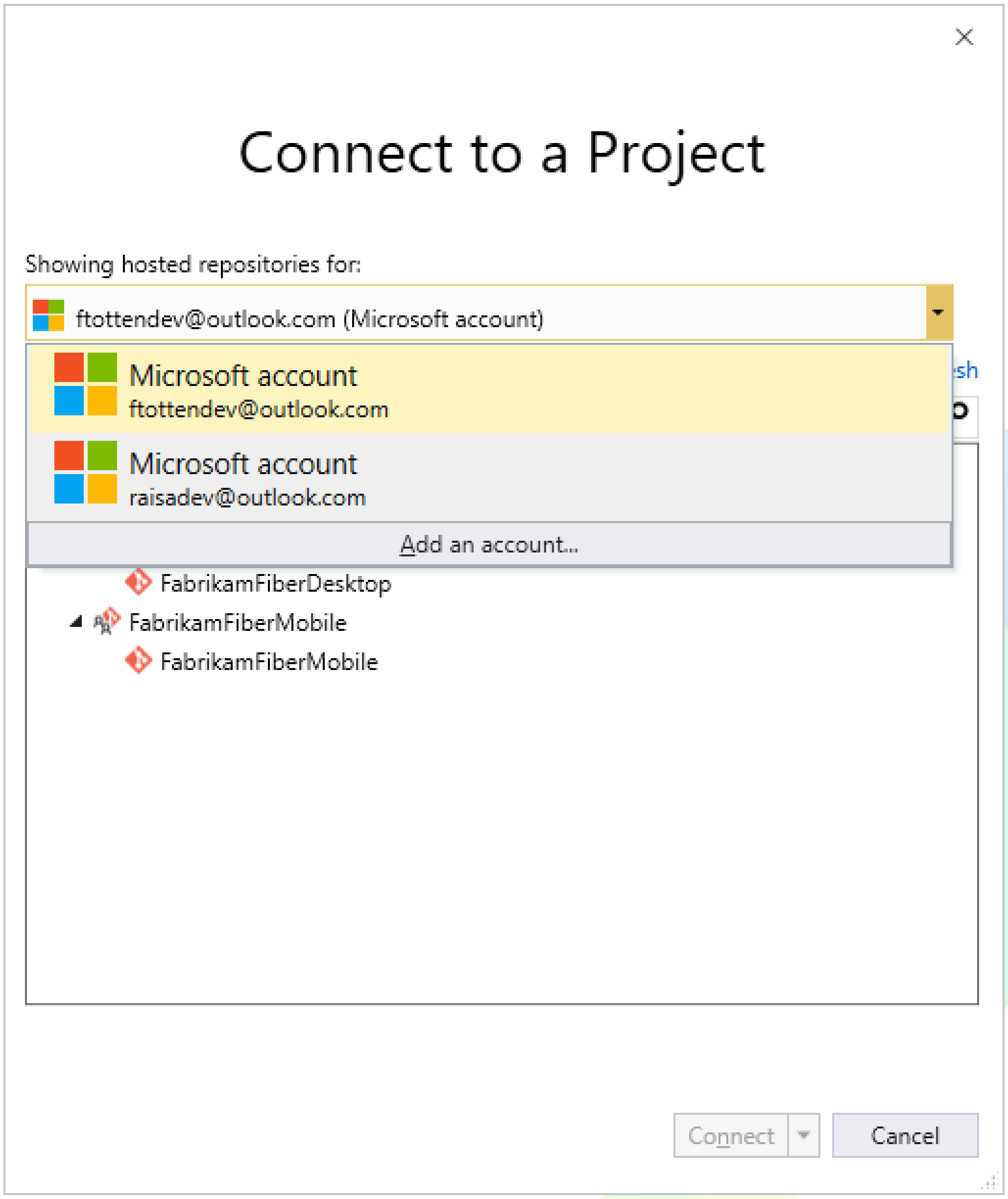
B. Connect to a project in Azure DevOps

If you're not a member of an Azure DevOps security group, get added to one. Check with a team member. You'll need the names of the server, project collection, and project to connect to.

Select the **Manage Connections** button in Team Explorer to open the **Connect** page. Choose **Connect to a Project** to select a project to connect to.

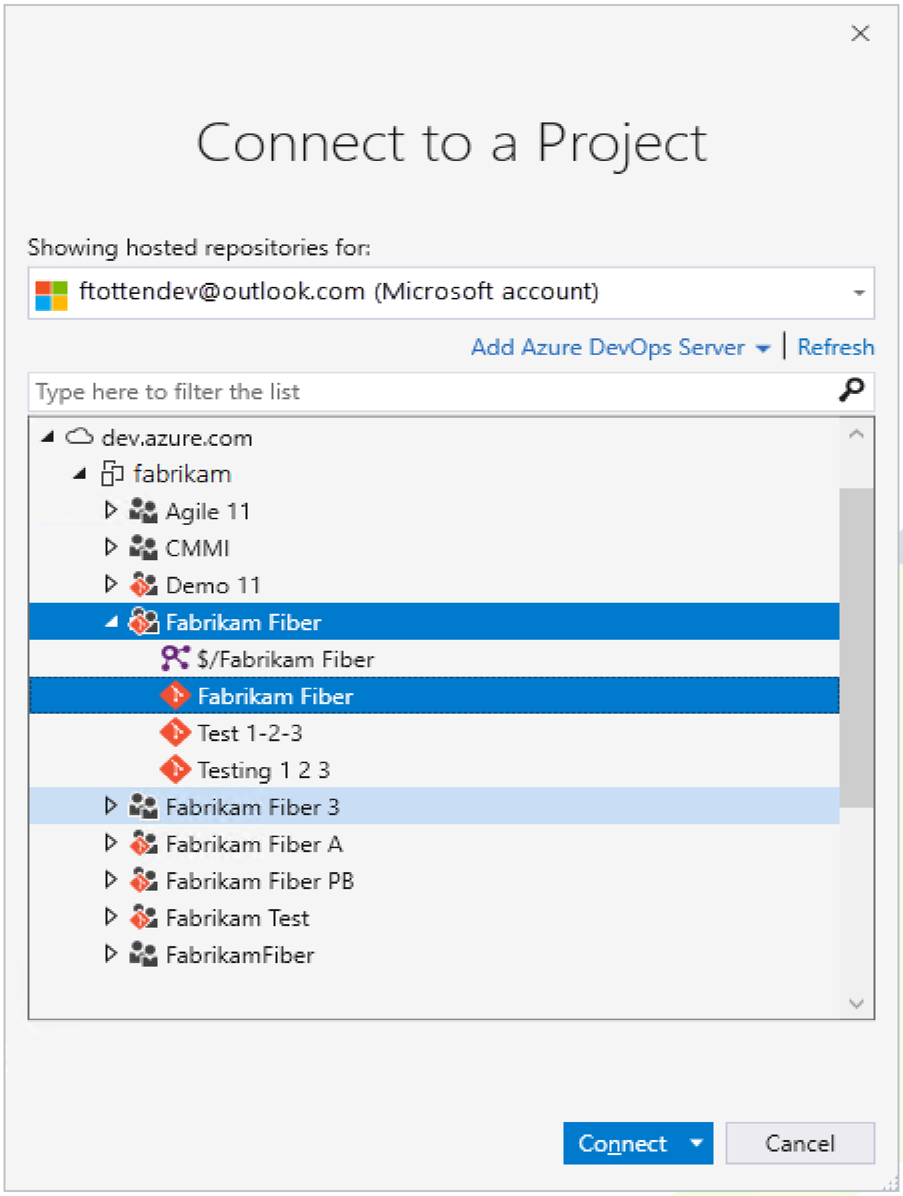


Select a different user or select Add an account to access a project using different credentials.



Sign in using an account that is associated with an Azure DevOps project, either a valid Microsoft account or GitHub account.

Connect to a Project shows the projects you can connect to, along with the repos in those projects.

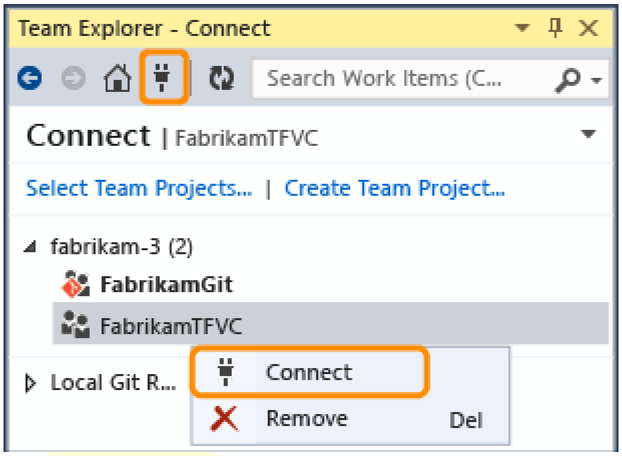


Select **Add Azure DevOps Server** to connect to a project in Azure DevOps Services. Enter the URL to your server and select **Add**.

Select a project from the list and select **Connect**.

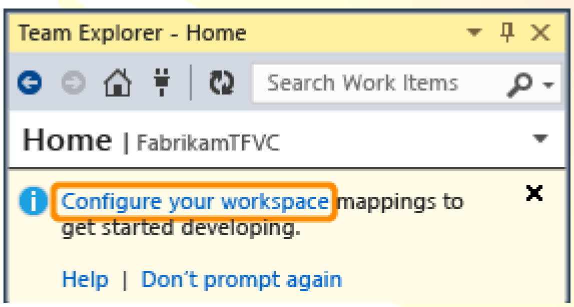
C. Create a workspace and get the code

From Visual Studio, go to the Team Explorer Connect page (Keyboard: Ctrl + 0, C) and then connect to the project.

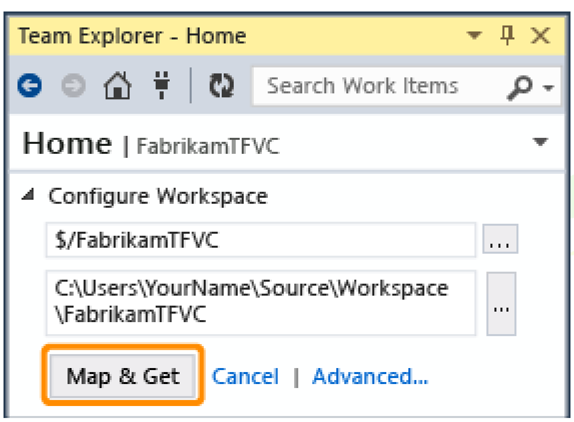


(If the project you want to open is not listed, choose **Select Projects** and then connect to the project.)

Map the project to a folder on your dev machine.

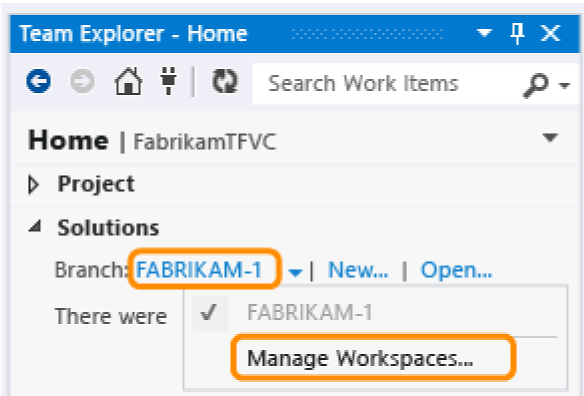


Map the workspace and get your code.

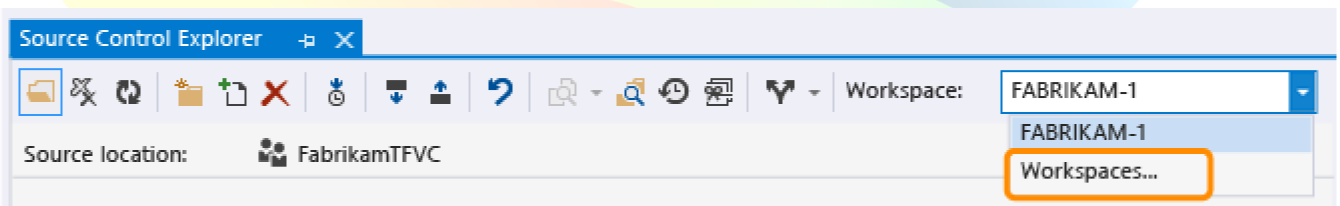


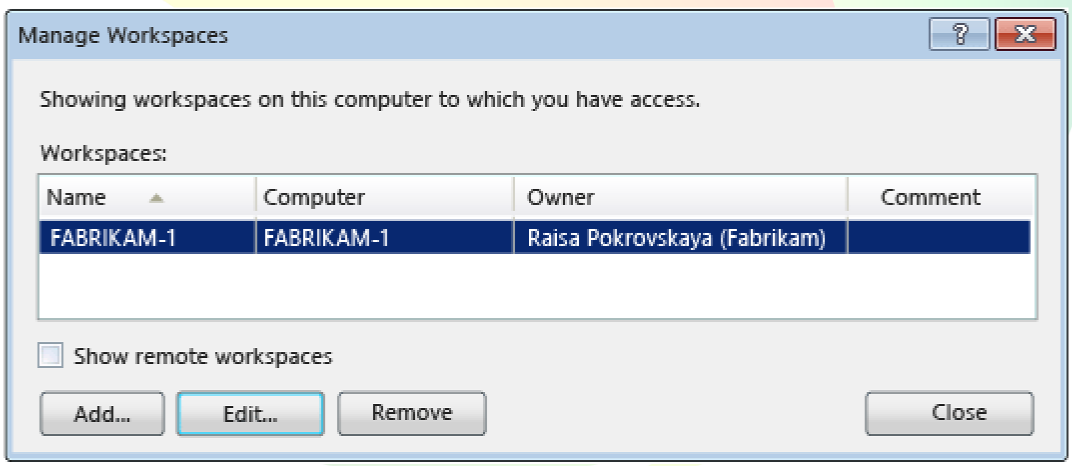
D. Add, edit, or remove a workspace

After you have connected to the project (Keyboard: Ctrl + 0, C), you can manage your workspaces from the Team Explorer home page (Keyboard: Ctrl + 0, H)



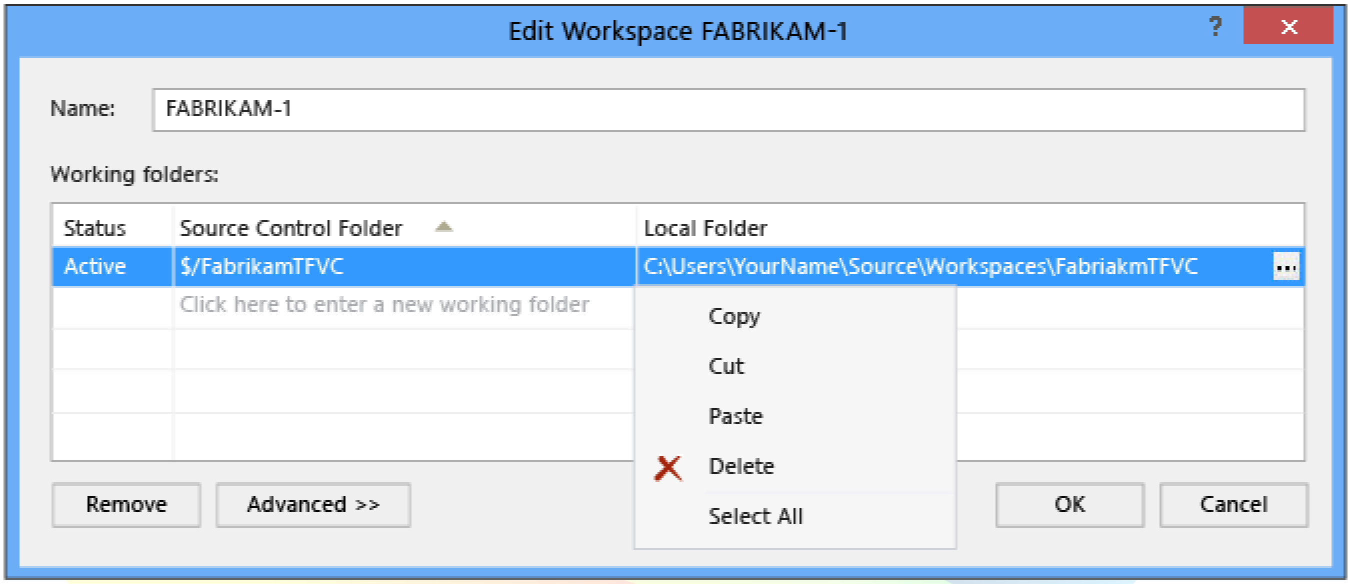
or from Source Control Explorer.





Choose **Show remote workspaces** if you want to view all the workspaces you own (including those on other computers).

After you choose **Add** or **Edit** you can modify working folders in a new or an existing workspace.



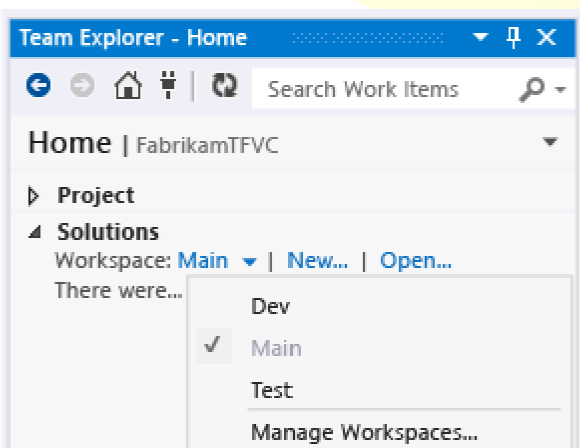
If you want to remove the workspace, before you do so, make sure there are no pending changes (Keyboard: Ctrl + 0, P). If you have pending changes, you can either check them in or shelve them.

Switch workspaces

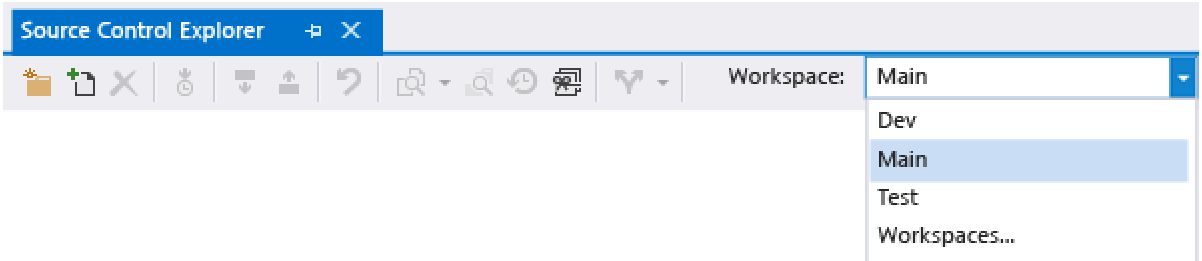
When you switch from one workspace to another, to avoid confusing yourself, make sure you switch to the same workspace in both **Team Explorer** and **Source Control Explorer**.

Connect to the project (Keyboard: Ctrl + 0, C).

On the home page (Keyboard: Ctrl + 0, H) choose the workspace you want to use.



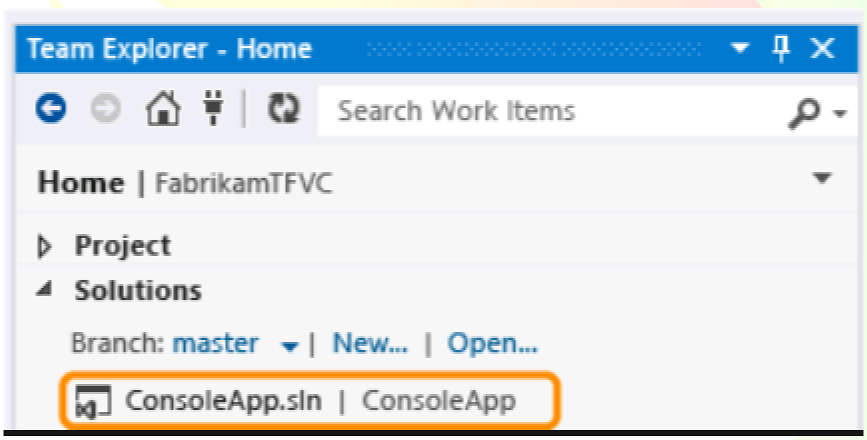
In Source Control Explorer, choose the workspace you want to work in.



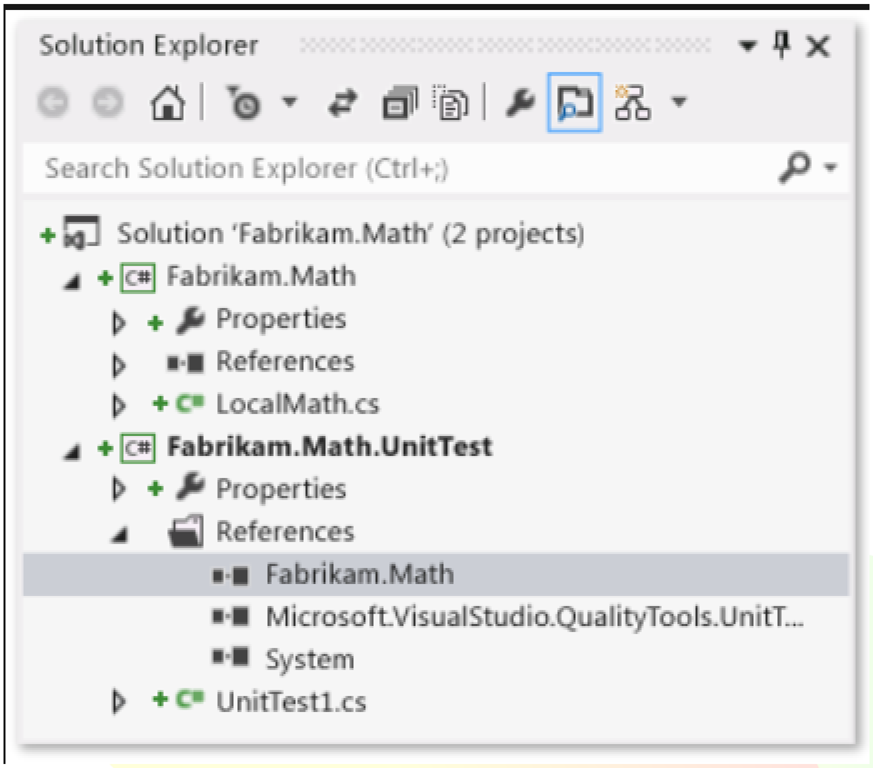
E. Develop code and manage pending changes

Most changes that you make to your files are queued as pending changes. As you work, you can organize, manage, and get details about what you've changed.

From the team explorer home page (Keyboard: Ctrl + 0, H), you can begin coding in a new or in an existing solution.



After you open your solution, open the solution explorer (Keyboard: Ctrl + Alt + L).

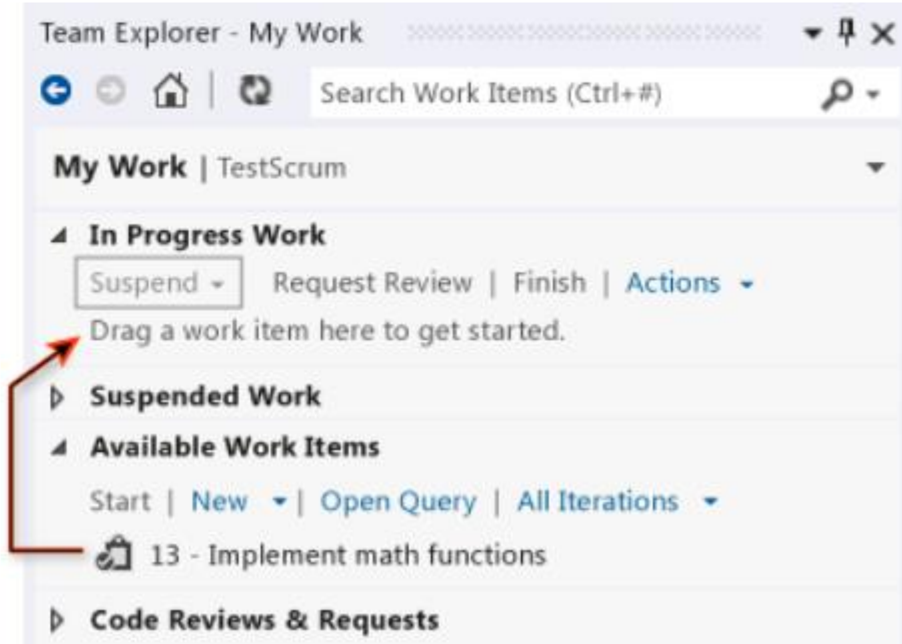


When you open and modify a file from the solution explorer, the file is automatically checked out for you. Icons appear to indicate which files you have not changed , those you have checked out , and those you have added to the solution .

If you're working in a solution that contains a lot of files, you'll probably find it convenient to filter the solution explorer to show only the files you have changed (Keyboard: Ctrl + [, P).

Use the My Work page to manage your work

If you're using Visual Studio Premium or Visual Studio Ultimate, you can use the My Work (Keyboard: Ctrl + 0, M) page in the team explorer to manage your work.



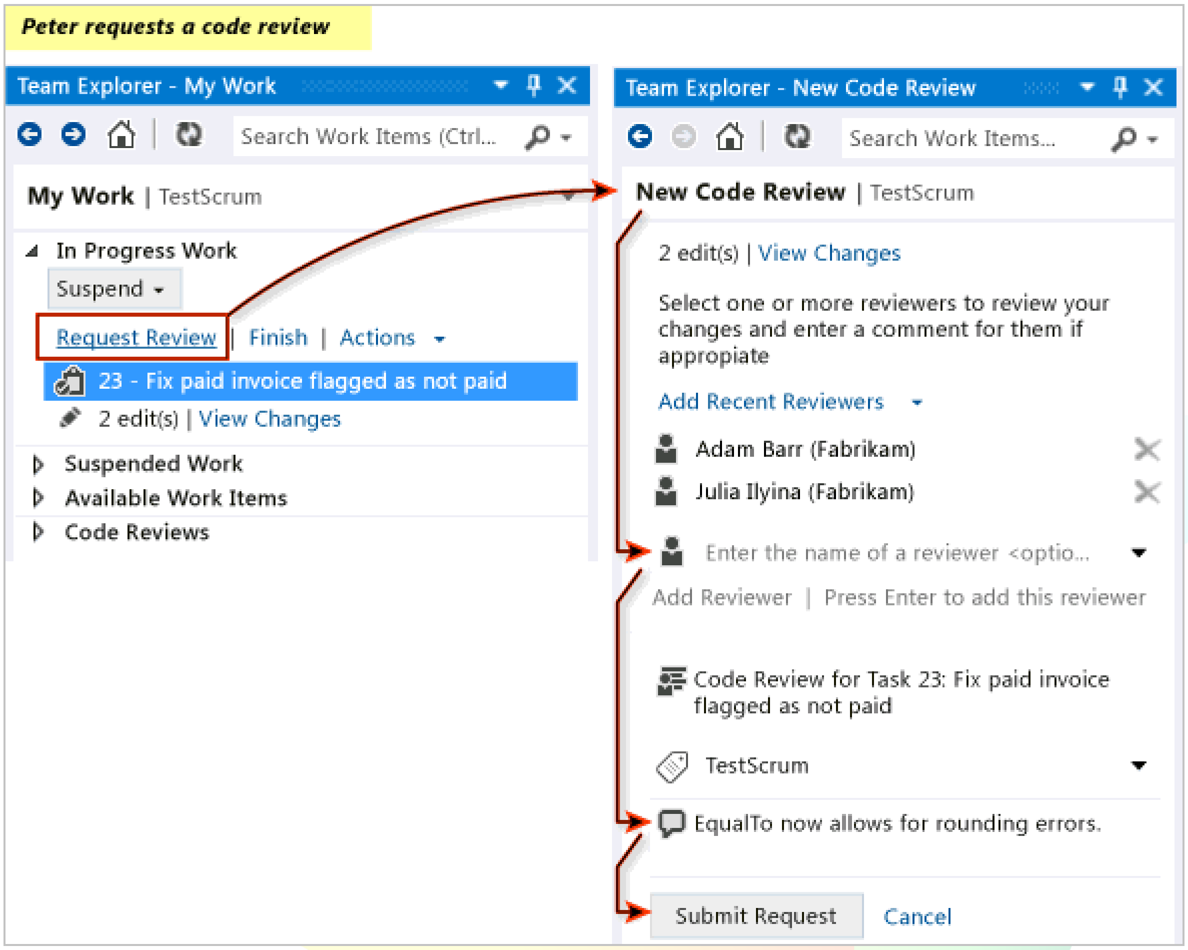
You can use My Work to: Track your work against work items

Suspend and later resume your work (including file changes, associated work items, and Visual Studio state such as window positions and breakpoints)

F. Request a code review

Use code reviews to increase overall code quality and reduce the risk of creating more bugs.

To request a code review



In **Team Explorer**, on the **My Work** page, choose **Request Review**.

The **New Code Review** page appears.

Specify one or more reviewers.

Specify the name of the review.

Specify the area path.

Specify a comment to your reviewers.

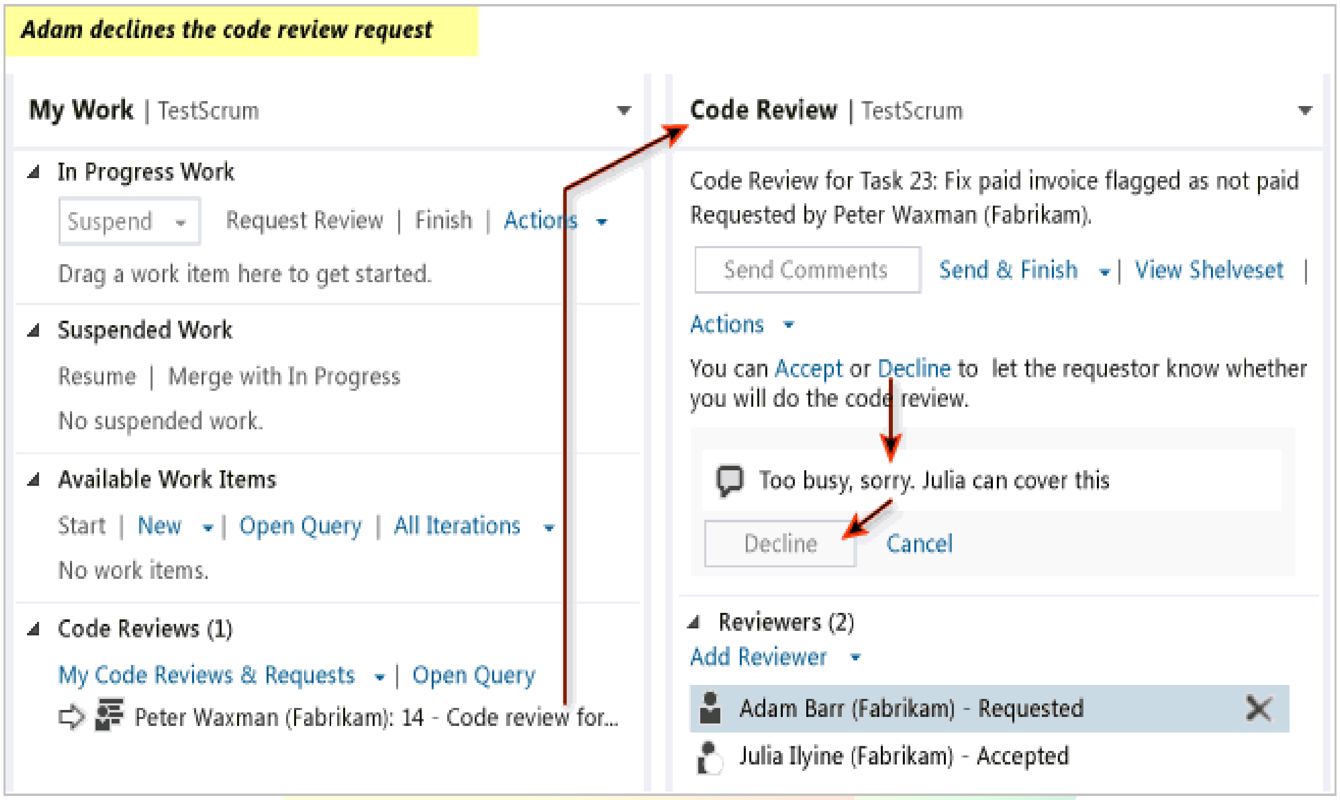
Choose **Submit Request**.

The reviewers will be notified of the request by email.

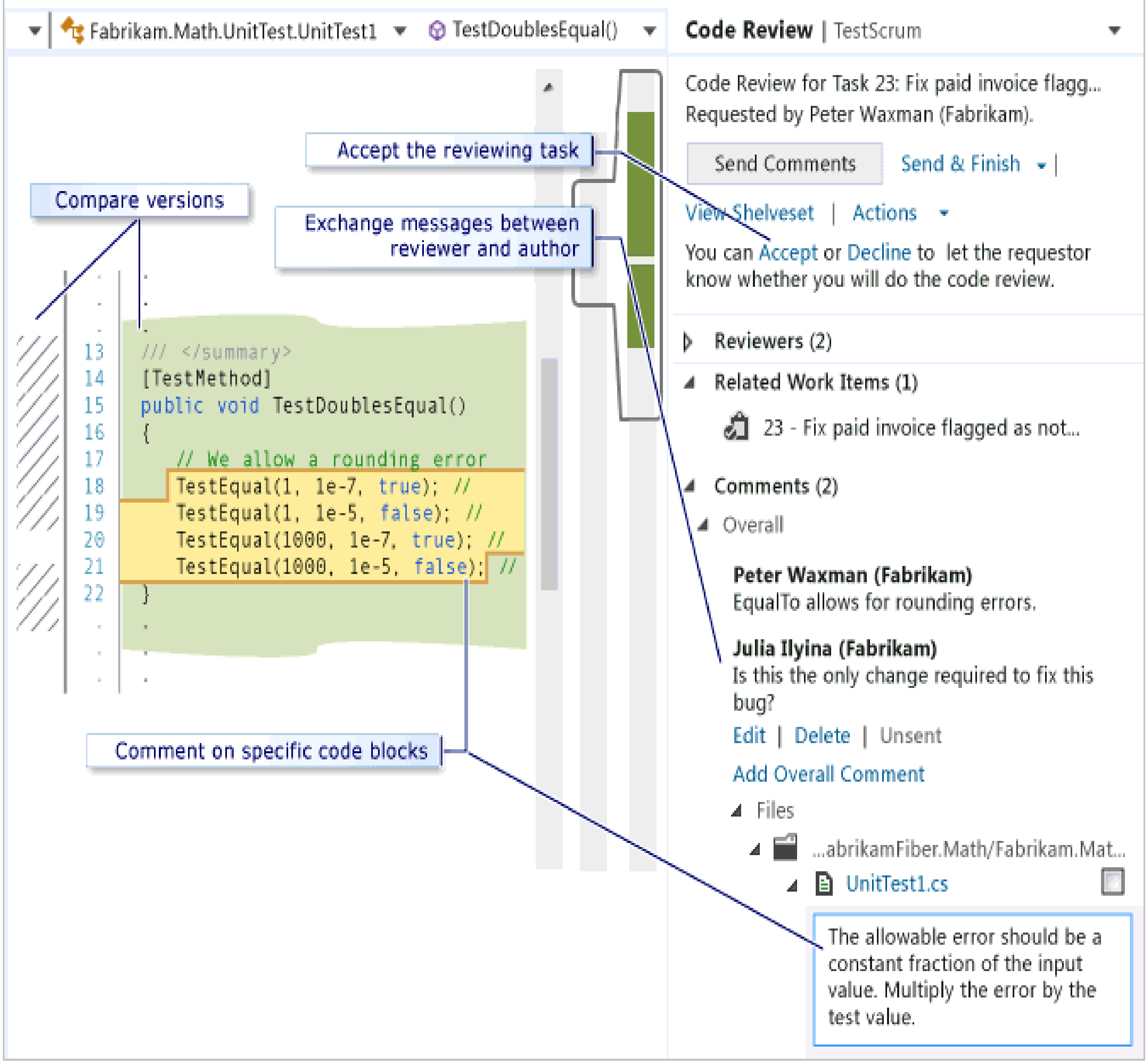
You can also request a code review of suspended work, a shelveset, or a changeset. To see a list of changesets, open **Source Control Explorer** and choose the **History** button.

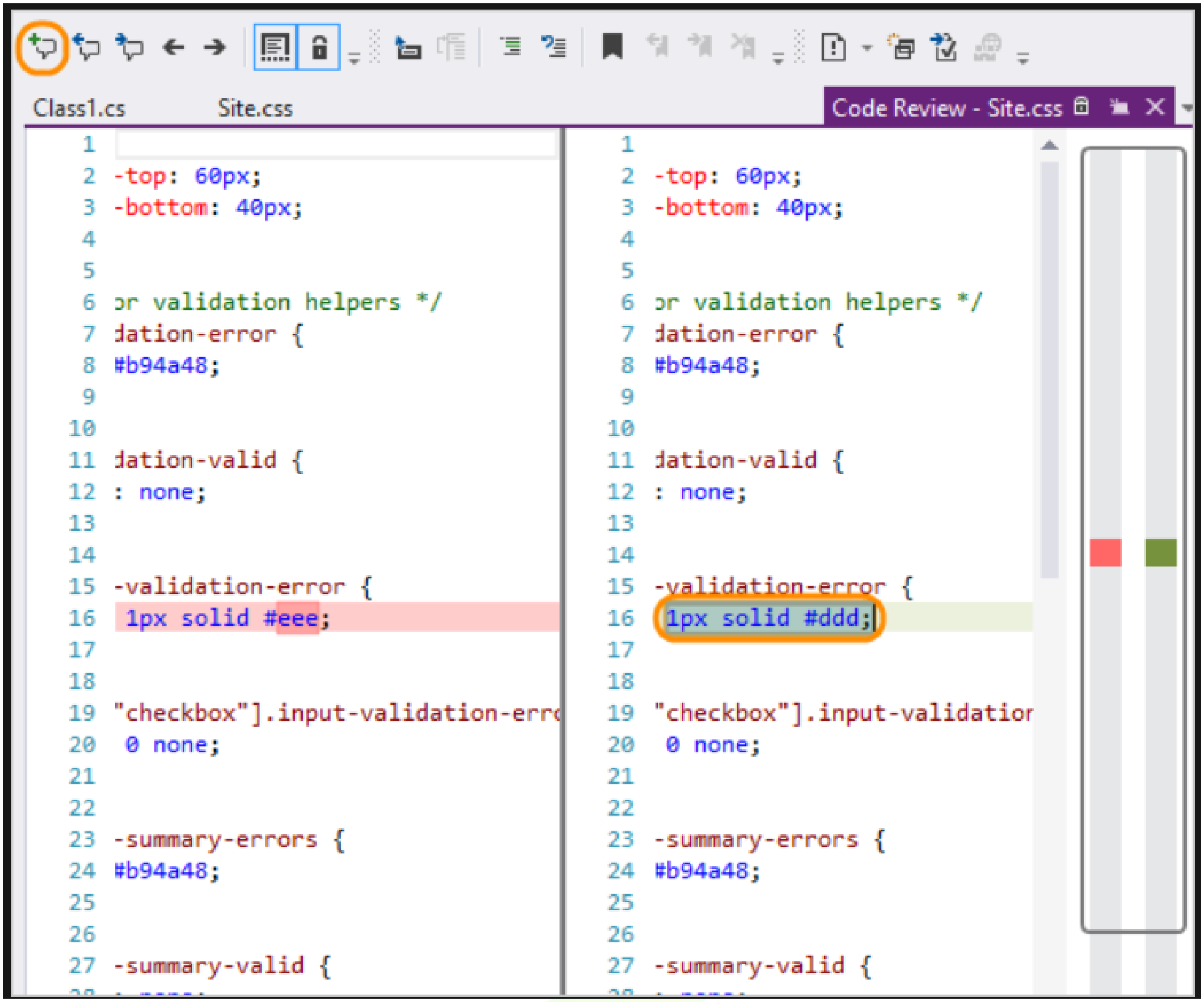
G. Accept or decline a code review

User will receive the code review request and accepts it. User reviews the code, writes some comments at the file- and code-block levels, and then sends the code review back to developer User. To perform a code review



Julia accepts the code review request





In **Team Explorer**, on the **My Work** page, go to the **My Code Reviews & Requests** section and open the request.

On the **Code Review** page, you can:

Choose **Accept** or **Decline** to notify the author whether you will perform the review.

Choose **Add Reviewer** to add other reviewers to the code-review request.

View the changes to each file that has been updated for this work item.

Expand **Comments** to discuss the changes with the author and other reviewers.

Choose **Add Overall Comment**

-or-

Select a block of code and then choose **Add Comment** from the shortcut menu.

Choose **Send Comments** to make your contributions visible to the author and other reviewers.

Choose **Send and Finish** to complete your review, indicating whether the code needs more work.

Respond to a code review

Peter receives and responds to the code review from Julia.

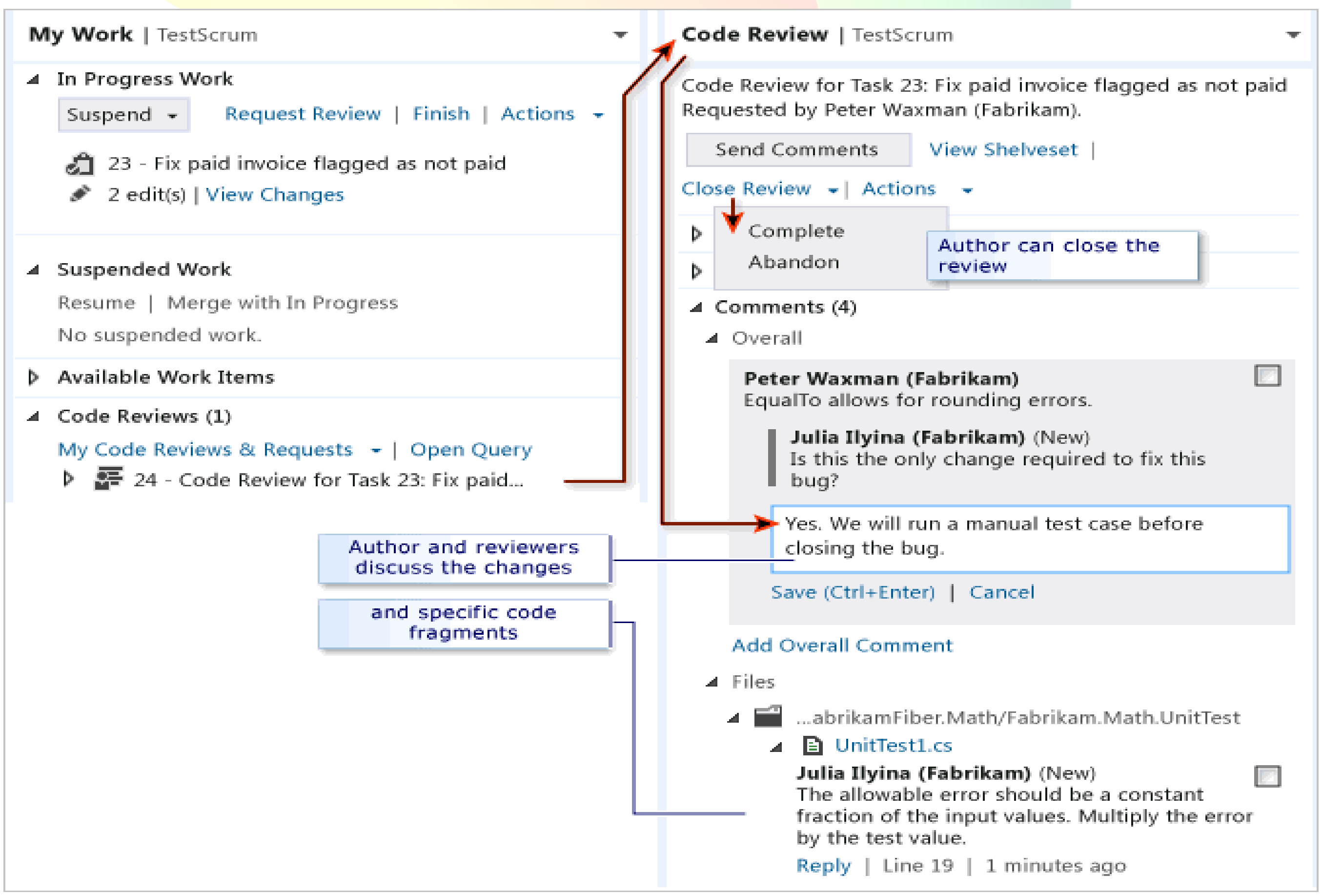
To respond to a code review

The reviewers and author of the code can exchange comments as often as they like. The review ends when the author closes it. With each contribution to the discussion, the other participants will be notified by email.

H. Respond to a code review

To respond to a code review

The reviewers and author of the code can exchange comments as often as they like. The review ends when the author closes it. With each contribution to the discussion, the other participants will be notified by email.



1. In **Team Explorer**, on the **My Work** page, go to the **Code Reviews & Request** section and double-click the request.

You can also open the shortcut menu for the request and choose **Open**.

2. Read the comments and reply to them as needed. To reply to a comment, choose **Reply**, enter your comment in the box that appears, and then choose **OK**. To send your comments, choose **Send Comments**.

3. To view a file and see the code-blocks that have comments, or to edit a file, go to the **Comments** section. In the **Files** sub-section, open the shortcut menu for the file and choose either **Compare (Read-Only)** or **Edit File**.

4. When you and the other reviewers finish responding to each other's comments and you are ready to close the review, click **Close Review**, and then choose either:
    - **Complete** to indicate that the review is finished.
    - -or-
    - **Abandon** to indicate you are canceling the review.